

CS673 Software Engineering**Team 2 - HealthNest****Software Test Document**

Your project Logo
here if any

<u>Team Member</u>	<u>Role(s)</u>	<u>Signature</u>	<u>Date</u>
Deven Shah	Team Lead, Requirements Lead		
Stephanie Wang	QA Lead, Design & Implementation Lead	<i>Stephanie Wang</i>	5/23/2026
Aaron Brown	Security Lead, Design & Implementation Lead		
Chamarr Auber	Team Lead, Configuration Lead	<i>Chamarr Auber</i>	5/24/2026

Revision history

<u>Version</u>	<u>Author</u>	<u>Date</u>	<u>Change</u>

[Testing Summary](#)

[Manual Tests Reports](#)

[Automated Testing Reports](#)

[Testing Metrics](#)

[References](#)

[Glossary](#)

● Testing Summary

In this section, you will summarize what was tested, who is involved in testing, when to test, testing techniques used, and testing result. You may have the following tests

- Unit Testing
- Integration testing
- System Testing
- Acceptance Testing
- Regression Testing

1.1 What Was Tested

The following features were in scope for Iteration 1:

- SCRUM-7 User Login: Sign-up, sign-in, sign-out, and token refresh for both Patient and Provider roles.
- SCRUM-59 Patient Home Dashboard: UI complete with mock data; backend connection in progress.
- SCRUM-26 Doctor Home Dashboard: UI complete with mock data; backend connection pending.
- SCRUM-5 Appointment Scheduler: Currently in development
- SCRUM-56 Biometric Authentication: Planned

1.2 Who Is Involved in Testing

- All developers: Each developer is responsible for ensuring their feature passes the CI/CD pipeline before requesting a merge.
- Configuration Lead: Reviews all pull requests, enforces commit naming conventions, and manages the CI/CD pipeline

1.3 When Testing Occurs

For Iteration 1, testing occurs through the CI/CD pipeline, which runs automatically on every pull request to main. It checks if the commit messages follow the Jira naming convention, runs ESLint as part of the frontend build to catch syntax errors and code

style issues, and confirms the build passes. PRs that fail are blocked from merging until fixed.

Formal manual testing began in Iteration 1 for the User Login feature. Additional manual testing for other features will continue in future Iteration once development is more complete.

1.4 Testing Techniques Used

- **Unit Testing** No unit tests have been written in Iteration 1. Unit tests will be implemented in future Iteration once the components are stable.
- **Integration Testing** No formal integration tests have been written in Iteration 1. However, the React frontend, FastAPI backend, and Supabase database were manually verified to work together through the login flow. Real user accounts were successfully created and stored in Supabase, which confirms all three layers are communicating correctly.
- **System Testing** No formal system test plan exists for Iteration 1. However, the full login flow was tested end-to-end by running the application locally via Docker. Testing for other features will be done in future iterations once development is more complete.
- **Acceptance Testing** The 7 manual test cases serve as acceptance tests for SCRUM-7 User Login. Each test case was written based on the requirements in the Jira user story and all 7 passed. Acceptance testing for other features will be done in future iterations.
- **Regression Testing** No formal regression tests exist yet. The CI/CD pipeline acts as a basic safety net where every pull request must pass the naming convention check and build before merging, which helps ensure new changes do not break what already works.

1.5 Testing Results

All 7 manual test cases for SCRUM-7 User Login passed. All 4 pull requests merged to main during Iteration 1 passed the CI/CD pipeline checks. One pull request initially failed due to a commit naming convention issue and was corrected before merging. No formal bugs have been logged in Jira as of Iteration 1.

• Manual Testing Report

In this section, you will give a detailed description of each manual test case performed and the result. If this is a previous You shall list what are existing tests developed in the previous semester and what are new tests developed currently.

Here is a sample template that can be used for each test case. For system tests or acceptance tests, you may also include some screenshots.

- Test case ID, name
- New or old:

- Test items: (what do you test)
- Test priority (high/medium/low)
- Dependencies (to other test case/requirement if any):
- Preconditions: (if any)
- input data:
- Test steps:
- Postconditions:
- Expected output:
- Actual output:
- Pass or Fail:
- Bug id/link: (this should link to your github issue id)
- Additional notes:

(You can use an additional spreadsheet for this section as well)

Test Case ID	TC01
Name	Patient Sign-Up with Valid Credentials
New or Old	New
Test Items	SCRUM-7 User Login — Sign-up flow for Patient role
Test Priority	High
Dependencies	Supabase authentication service must be running
Preconditions	Email address not previously registered in the system
Input Data	Full Name, MRN: MRN-00123456, Date of Birth: 05/22/2026, Sex at Birth: Prefer not to say, Email: BU email, Password: 6+ character password
Test Steps	1. Click 'Create an account as Patient' 2. Select Patient role 3. Fill in all required fields 4. Click 'Create patient account'
Postconditions	New user account created and stored in Supabase database
Expected Output	Success message
Actual Output	Green success message displayed: 'Account created. Check your email to confirm before signing in.'
Pass or Fail	PASS
Bug ID / Link	N/A
Additional Notes	User record appeared in Supabase authentication dashboard after sign-up

Test Case ID	TC02
---------------------	------

Name	Patient Sign-In with Valid Credentials
New or Old	New
Test Items	SCRUM-7 User Login — Sign-in flow for Patient role
Test Priority	High
Dependencies	TC01 must pass; user account must exist and email must be confirmed
Preconditions	User account created and email confirmation link clicked
Input Data	Email: BU email, Password: valid password, Role: Patient
Test Steps	1. Select Patient role 2. Enter valid email and password 3. Click 'Sign in as Patient'
Postconditions	User is authenticated and session is created in Supabase
Expected Output	User is successfully signed in and redirected to the Patient Dashboard
Actual Output	User signed in successfully and redirected to Patient Dashboard
Pass or Fail	PASS
Bug ID / Link	N/A
Additional Notes	None

Test Case ID	TC03
Name	Role-Based Routing — Patient Redirected to Patient Dashboard
New or Old	New
Test Items	SCRUM-7 User Login — Role-based routing after authentication
Test Priority	High
Dependencies	TC02 must pass
Preconditions	User signed in as Patient role
Input Data	Patient role selected at login
Test Steps	1. Sign in as Patient 2. Observe which page loads after successful login
Postconditions	User lands on Patient Dashboard
Expected Output	Application automatically routes Patient user to the Patient Dashboard after login
Actual Output	Patient Dashboard loaded immediately after sign-in with no manual navigation required
Pass or Fail	PASS

Bug ID / Link	N/A
Additional Notes	None

Test Case ID	TC04
Name	Sign-In with Wrong Password
New or Old	New
Test Items	SCRUM-7 User Login — Error handling for invalid credentials
Test Priority	High
Dependencies	TC01 must pass; user account must exist
Preconditions	Valid user account exists in the system
Input Data	Email: BU Email Password: incorrect password
Test Steps	1. Select Patient role 2. Enter valid email and an incorrect password 3. Click 'Sign in as Patient'
Postconditions	User remains on login page; no session created
Expected Output	Error message displayed indicating invalid credentials
Actual Output	Red error message displayed: 'Invalid login credentials'
Pass or Fail	PASS
Bug ID / Link	N/A
Additional Notes	Error message is clear

Test Case ID	TC05
Name	Sign-In with Empty Fields
New or Old	New
Test Items	SCRUM-7 User Login — Empty field validation on sign-in
Test Priority	Medium
Dependencies	None
Preconditions	User is on the login page
Input Data	Email: empty, Password: empty

Test Steps	1. Select Patient role 2. Leave email and password fields empty 3. Click 'Sign in as Patient'
Postconditions	Form does not submit; user remains on login page
Expected Output	Submission is prevented and user is prompted to fill in required fields
Actual Output	Browser native autocomplete dropdown appeared; form submission was prevented by browser-level validation
Pass or Fail	PASS
Bug ID / Link	N/A
Additional Notes	Validation is currently handled by browser native behavior. A future improvement could be to add a custom app-level error message such as 'Please enter your email and password' for a more consistent user experience.

Test Case ID	TC06
Name	Sign-Up with Password Too Short
New or Old	New
Test Items	SCRUM-7 User Login — Password length validation on sign-up
Test Priority	High
Dependencies	None
Preconditions	User is on the sign-up page
Input Data	Email: BU Email, Password: 'ab' (2 characters)
Test Steps	1. Click 'Create an account as Patient' 2. Fill in name, MRN, and email fields 3. Enter a password with only 2 characters 4. Click 'Create patient account'
Postconditions	Account is not created; user remains on sign-up page
Expected Output	Error message displayed indicating password is too short (minimum 6 characters)
Actual Output	Error message displayed: 'Please use at least 6 characters (you are currently using 2 characters).'
Pass or Fail	PASS
Bug ID / Link	N/A
Additional Notes	Password minimum length of 6 characters is enforced correctly by the application

Test Case ID	TC07
Name	Patient Sign-Out
New or Old	New
Test Items	SCRUM-7 User Login — Sign out flow for Patient role
Test Priority	High
Dependencies	TC002 must pass; user must be signed in
Preconditions	User is signed in as Patient and on the Patient Dashboard
Input Data	Authenticated Patient session
Test Steps	1. Sign in as Patient 2. On the Patient Dashboard, click the Sign Out button 3. Observe which page loads
Postconditions	User session is terminated; user is logged out of the system
Expected Output	User is signed out and redirected back to the Login page
Actual Output	User was signed out and redirected directly back to the Login page
Pass or Fail	PASS
Bug ID / Link	N/A
Additional Notes	Sign-out completes immediately with no delay or confirmation prompt

● Automated Testing Report

Describe briefly the automated testing you have done, including where the test code resides in your code repository, what test frameworks are used, and the screen shots or generated testing report.

No formal automated test suites have been implemented in Iteration 1.

The following checks run automatically through the CI/CD pipeline on every pull request to main:

- **Validate Jira Key:** Checks that every commit message starts with a Jira ticket ID. If it does not pass, the PR gets blocked
- **Frontend Build and Test:** Runs the frontend build to confirm everything compiles without errors. ESLint also runs as part of this step, checking all .js and .jsx files for syntax errors and code issues based on the rules set in frontend/eslint.config.js
- **Docker Build Check:** Confirms the full application builds correctly inside Docker.
- Below is a screenshot showing all checks passed:

8 checks passed		
✓	Validate Jira Key	Details
✓	Validate Jira Key	Details
✓	Frontend Build and Test	Details
✓	Frontend Build and Test	Details
✓	Docker Build Check	Details
✓	Docker Build Check	Details
⌚	Release Build	Details
⌚	Release Build	Details

● Testing Metrics

In this section, you shall report any metrics used for the evaluation, e.g. # of test cases, test coverage, defects rate, etc.

Metric	Value	Notes
Total manual test cases	7	TC01-TC07 for SCRUM-7 User Login
Manual tests passed	7	All 7 test cases passed
Manual tests failed	0	No failures recorded
Test pass rate	100%	All manual test cases passed
Defects found during testing	0	No defects formally filed
Code coverage	N/A	To be added once unit tests are in place
CI/CD checks passed	5 of 5	All pr passed pipeline checks

● AI usage Log

(This table summarizes AI contribution in this document)

Section	Who	AI contribution (%)	Description	Verified by
Testing Summary	Stephanie	35%	I used Claude to help structure and word each subsection based on my meeting notes, code review, and what the team discussed	
Automated Testing	Stephanie	30%	I used Claude to help write up the CI/CD checks based on	

Section	Who	AI contribution (%)	Description	Verified by
			what I found/understand in the GitHub Actions pipeline.	

- References
- Glossary